SME and Accelerate on Apple M4 Pro

One complete SME implementation and one vendor-library reference.

Understand the matrix operation. Deliver its operands efficiently. Measure the complete product.

Native C intrinsics · FP32 · square matrices
SME2 · 512-bit streaming vector length

What this deck covers

Outer products and ZA
Final packing and kernels
Blocking and edge handling
Throughput and speedup
Reproducible measurements

Architecture diagrams explain the design; the charts show measured local results.

SME is an instruction set; Accelerate is a library

SME means Scalable Matrix Extension.

Arm SME / SME2

Streaming execution mode
Z vectors and predicates
ZA matrix accumulators
SME2 vector-group loads

Apple Accelerate

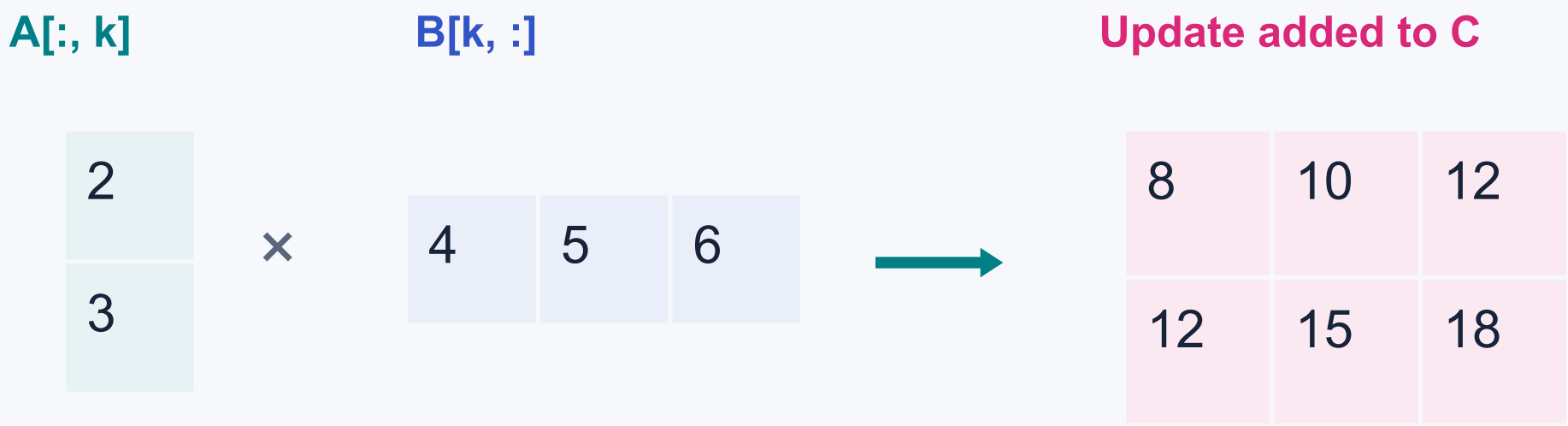
BLAS interface: cblas_sgemm
Computes C = A × B
Library selects its internal path
Forced to one thread here

This is an implementation comparison, with the same inputs and timing boundary.

Sources: Arm ACLE [1] · Apple Accelerate [3] · src/accelerate.c

One outer product updates many output values

The full product is a sum over K: C += A[:, k] × B[k, :].



16 FP32 lanes × 16 lanes = 256 updates per FMOPA on this machine.

256 fused multiply-adds = 512 nominal FLOPs per FP32 FMOPA.

Sources: Arm ACLE [1] · Hello SME [2] · teaching example constructed for this deck

ZA exposes four logical FP32 tiles at 512-bit SVL

The runtime checks for 16 FP32 lanes before executing the SME backend.

A Z vector: 16 × 32-bit values

One ZA tile: 16 × 16 FP32
Four tiles: 4 × 1024 bytes
Total ZA storage: 4096 bytes

Predicates select active lanes.
The size contract uses multiples of 32.

ZA0.S
16 × 16
256 FP32

ZA1.S
16 × 16
256 FP32

ZA2.S
16 × 16
256 FP32

ZA3.S
16 × 16
256 FP32

Both kernels interleave updates across all four logical ZA tile accumulators.

Sources: Arm ACLE [1] · Hello SME [2] · src/sme.c · main.c

One SME backend adapts its data path to matrix size

The two kernels are complementary parts of the final implementation.

N ≤ 512

A: transpose in ZA
B: original row-major input
Output tile: 32×32
No B preparation buffer

N > 512

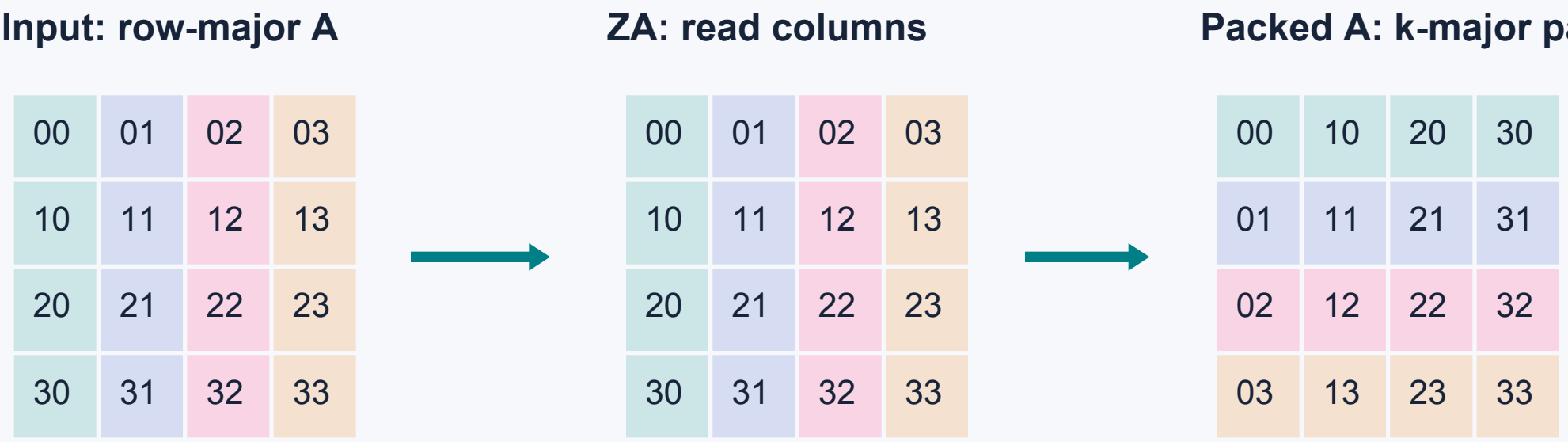
A16 / B64 packed panels
Output tile: 16×64
KC=2048, MC=256, NC=512
Final 32 columns handled in place

Input and output matrices remain N×N row-major FP32 arrays.

Sources: src/gemm.h · src/sme.c · src/pack.c

Build contiguous A panels by reading ZA columns

Horizontal tile loads followed by vertical-group reads express the transpose.



Actual 16×16 block: 16 row loads → 4 column-group reads → 4 grouped stores.

A[k*16 + lane] supplies 16 consecutive output-row values for each k.

Source: src/pack.c · intrinsic semantics: Arm ACLE [1]

For N≤512, B can be consumed without packing

Each row already contains the consecutive columns needed by the kernel.

One 32×32 output tile

ZA0.S
A0 × B0

ZA1.S
A0 × B1

ZA2.S
A1 × B0

ZA3.S
A1 × B1

Per k

Load A0 and A1 from packed A
Load B0 and B1 from original B
Issue four FMOPA instructions
Keep the tile in ZA until K is complete

The four tile updates perform 2048 nominal FLOPs for each k.

Source: src/sme.c: direct_b · src/pack.c: pack_inputs

The 16×64 kernel reuses one A vector four times

SME2 loads four contiguous B vectors as a group.

A: one vector
16 rows

A

B: four vectors / 64 columns

B0 · 16 lanes	B1 · 16 lanes	B2 · 16 lanes	B3 · 16 lanes
ZA0.S	**ZA1.S**	**ZA2.S**	**ZA3.S**
A × B0	A × B1	A × B2	A × B3

Per k: 4 FMOPA instructions · 2048 FLOPs · 320 operand bytes, excluding C.

Grouped loads, short live ranges, and matching packed panels work together.

Source: src/sme.c: tile_16x64 · src/pack.c

Load, consume, and retire the tuple in one iteration

The final loop is unrolled four times; helpers share the caller’s ZA state.

```
#pragma clang loop unroll_count(4)
for (size_t k = 0; k < kc; ++k) {
  svfloat32_t va = svld1_f32(pg, a+k*16);
  svfloat32x4_t vb = svld1_f32_x4(pg4, b+k*64);
  svmopa_za32_f32_m(0, pg, pg, va, svget4_f32(vb,0));
  svmopa_za32_f32_m(1, pg, pg, va, svget4_f32(vb,1));
  svmopa_za32_f32_m(2, pg, pg, va, svget4_f32(vb,2));
  svmopa_za32_f32_m(3, pg, pg, va, svget4_f32(vb,3));
}
```

Within each iteration

Load A and B
Consume all B vectors
Reuse A four times
End tuple lifetime

Inspect the generated code on the compiler actually used for the benchmark.

Source: src/sme.c: tile_16x64 · generated instructions audited by build.py

Bound the working panels and handle the last 32 columns

Compute blocks: KC=2048, MC=256, NC=512. B packing chunks: 32×256.

A panel

256 × 2048 FP32
2 MiB logical footprint
16-row micro-panels
No padding

B panel

2048 × 512 FP32
4 MiB logical footprint
64-column micro-panels
Only the final tail is padded

C output

Original N×N allocation
16×64 output regions
Skip nonexistent columns
Reload for later K blocks

At N=4096, KC=2048 needs two K passes per output region.

Edge handling belongs to the same final kernel; no legacy fallback is retained.

Sources: src/gemm.h · src/pack.c · src/sme.c

Compare the same complete row-major matrix product

Apple M4 Pro · 48 GB · 48 jobs / 336 samples

Timed

Required packing on every SME call
The complete product C = A × B
All C loads and stores
Accelerate’s public SGEMM call

Protocol

One thread for both backends
Identical deterministic inputs
3 rounds × 7 batch-average samples
Cases and backends randomized

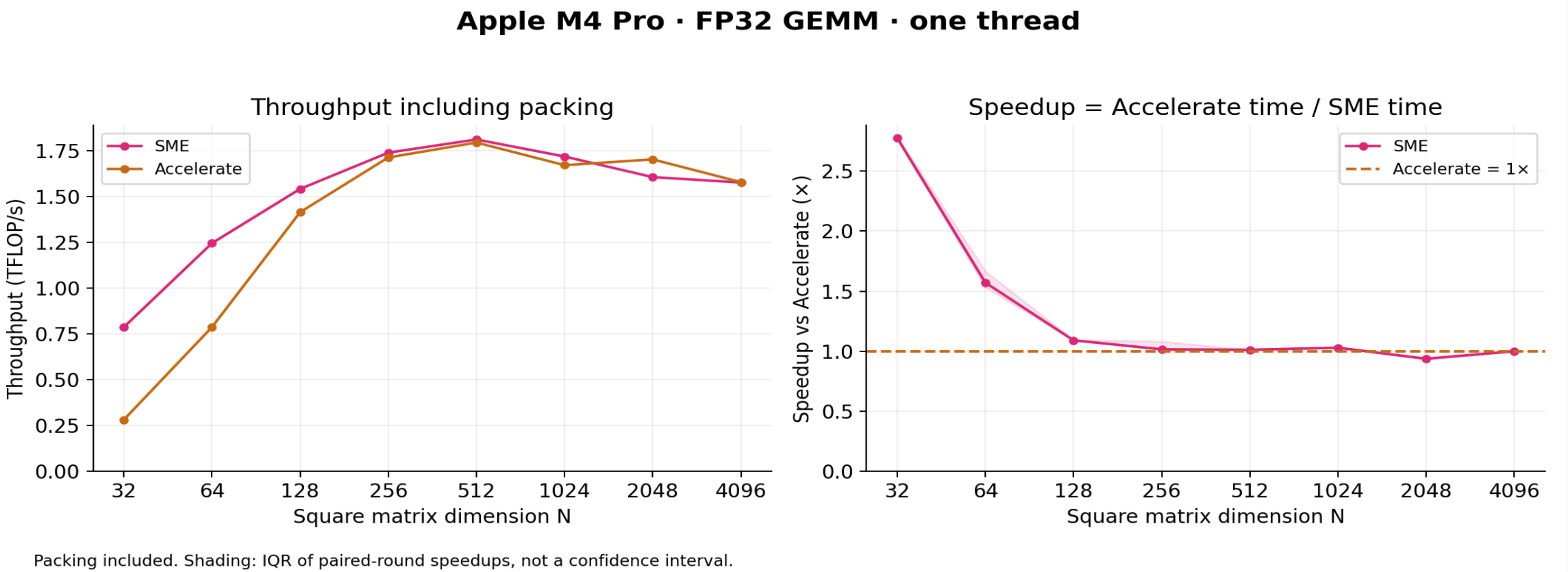
Higher throughput is better. Speedup above 1× means SME is faster than Accelerate.

Warm reused inputs, one calling thread: these measurements are not chip-wide peak throughput.

Sources: main.c · benchmark.py · examples/m4-pro/environment.json · Apple BLAS threading [4]

SME versus Accelerate: throughput and speedup

All required packing is included. Speedup = Accelerate time / SME time.



Shading shows variability across paired rounds; absolute throughput remains essential context.

Read the ratios alongside absolute performance

Packing included · GFLOP/s from pooled medians · speedup from paired rounds

N	SME GFLOP/s	Accelerate GFLOP/s	SME / Accelerate
128	**1542**	1415	1.09×
512	**1812**	1795	1.01×
1024	**1719**	1672	1.03×
2048	**1606**	1703	0.94×
4096	**1577**	1579	1.00×

One final SME backend, one Accelerate reference, and no experimental implementations.

Build, verify, benchmark, and plot

The project keeps the implementation and its measurement path small.

```
python3 build.py
python3 check.py
python3 benchmark.py --suite full --output results/run1
python3 plot.py results/run1
```

```
# Existing results only; no benchmark execution:
python3 slides/build_slides.py
```

Evidence

ASan / UBSan tests
Independent reference
Input and binary hashes
Disassembly checks
Measured cleanup validation

The final SME design combines efficient arithmetic, packing, and data reuse.

Sources: README.md · build.py · check.py · benchmark.py · plot.py · validation/cleanup/

REFERENCES

Follow the implementation back to primary sources

Architecture/API references and the exact current source files.

[1] Arm C Language Extensions

Streaming state, ZA attributes, intrinsics

[2] Hello SME / University Jena

FMOPA, SVL and logical tile views

[3] Apple Accelerate

BLAS and runtime implementation selection

[4] Apple BLAS threading

Explicit single-thread execution control

Local reading order

src/gemm.h → interface and workspace
src/pack.c → final A/B preparation
src/sme.c → final SME kernels
src/accelerate.c → library reference
examples/m4-pro/ → current results
validation/cleanup/ → equivalence evidence

Every plotted performance value can be traced to the saved benchmark records.